> **Facilitator answer key — Holocron capstone worked solution.** Facilitator-only — lives in the facilitator folder of every repo; do not hand to participants. Part of the internally-consistent set in this folder (see `README.md`).

DP-light — Holocron release 1 (delivery tool: Azure DevOps)

Scope: the release-1 slice — access roles, product setup, create/edit/publish strings, version history + audit, search, delivery to consuming apps, request/complete review, translation variants. Out: aliases, advanced reviewer config, export/import, rollback.

1. Outcome map (compressed)

Outcomes move the world; outputs (shipped stories) are just how we move them. Each leading indicator is measurable within 7 days; each guardrail exists so we don't game the indicator.

#	Outcome (user-/business-visible)	Leading indicator (≤7 days)	Guardrail (don't game it)
O1 — primary	Time to change a published UI string drops from days/weeks (code deploy) to minutes (no deploy)	p50 author→consumed lead time for a text change ≤ 15 min, tracked daily	Change must reach consuming apps within propagation SLO (≤ 60s) — "fast to publish, still stale in prod" doesn't count
O2 — supporting	Engineering hours spent on text-only changes fall toward zero	# of text-change code deploys / week trends to 0; # of strings edited by non-engineers ↑	Reject-on-publish rate and post-publish edit ("hot-fix a typo") rate stay flat — speed can't come from skipped care
O3 — supporting	Consuming apps adopt Holocron instead of hardcoding	# of apps live on delivery API ↑; # of namespaces served ↑ week over week	New apps onboarded still hardcode < X% of strings — adoption must be real integration, not a stub
O4 — counter-outcome (watch)	We are NOT trading speed for governance/quality	Strings requiring legal review that published *without* it = 0	— (this is the guardrail line for the whole plan)

2. Backlog skeleton (ADO shape)

One **Epic** → four **Features** → representative **User Stories**. ADO fields shown: Title · State · Story Points (SP) · Area Path · Tags. Tag `CSn` traces each story to its Capability Story in the reference PRD; tag `R1` = release-1 slice.

Epic: Holocron release 1 — governed string lifecycle with no-deploy publishing · Area Path `Holocron\R1` · Tag `R1`

Feature	User Story (ADO)	State	SP	Tags
F1 Authoring	US-101 As a Content Owner I create/edit a source string in a product I own so I can change UI text without engineering	New	5	`R1,CS3,CS4,Auth`
	US-102 As a Content Owner I publish a string so the change is live, and every version is retained with author/timestamp	New	5	`R1,CS4,CS5,Auth`
	US-103 As an Admin I set up a product + assign Content Owners so scope and permissions are enforced	New	3	`R1,CS1,CS2,Acce`
F2 Review	US-201 As a Content Owner I request review on a string so legal/compliance can approve before publish	New	3	`R1,CS11,Review`
	US-202 As a Reviewer I complete a review (approve/reject + note) so the decision is recorded in the audit trail	New	3	`R1,CS12,Review,`

Feature	User Story (ADO)	State	SP	Tags
F3 Delivery	US-301 As a consuming app I fetch strings by app+namespace over the delivery API so I stop hardcoding	New	8	`R1,CS7,Delivery`
	US-302 As a consuming app I always get an en-US fallback when a locale value is missing so I never render a blank	New	3	`R1,CS7,Delivery`
	US-303 As any employee I search strings across products so I can find the key to change	New	3	`R1,CS6,Search`
F4 Translation	US-401 As a Content Owner I add/edit a locale variant of a published key so consuming apps serve localized text	New	5	`R1,CS15,Transla`

Slicing note: immutable keys and the Redis read-cache are cross-cutting technical constraints (from TCD/TMD-light), realized inside US-102/US-301, not separate stories.

3. Tracking plan (compressed, ADO)

Output → outcome → leading-indicator triples. Each indicator has an ADO home (a saved **query**, a **dashboard chart**, or the board **CFD** — cumulative flow diagram).

Output (what shipped)	Outcome served	Leading indicator	ADO instrument
US-102 publish + audit	O1 time-to-change ↓	p50 lead time author→consumed	Analytics **lead-time v** `Holocron\R1` boar tile
US-301/302 delivery API	O3 adoption ↑	# apps live · # namespaces served	Saved **query** `Tag=D` `AND State=Done`, p chart of distinct app I
US-201/202 review flow	O4 governance held	strings published bypassing required review = 0	Saved **query** on cust `RequiredReview=Y` `PublishedWithoutF` (must return 0)
Whole F1–F4 flow	O2 eng hours ↓	text-change code deploys/week → 0	Manual weekly metri dashboard; **CFD** wat review-column WIP b

Cadence: weekly 30-min delivery review off the ADO dashboard (indicators + CFD flow health); monthly outcome review against O1–O3 targets with the counter-outcome O4 as the standing first agenda item.

4. Value stream (compressed)

Flow for one string change. **PT** = hands-on process time; **LT** = elapsed lead time incl. wait. The dominant queue is the **review-approval wait**.

Step	PT	LT	Note
Author edits string (US-101/102)	5 min	5 min	in ADO/Holocron, no code
Wait: review queue (US-201)	—	**~1–2 days**	← obvious queue; only for review-required strings
Reviewer approves (US-202)	5 min	5 min	legal/compliance decision, audited
Publish (US-102)	1 min	1 min	immutable version written
Propagate via Redis cache invalidation	~0	≤ 60s	propagation SLO
Consumed by app (US-301)	~0	≤ 200ms p95	delivery SLO

Process time ≈ **11 min**; lead time ≈ **1–2 days**, ~99% of it the review-approval queue. Non-review strings already flow author→consumed in minutes — the queue is the whole gap between O1's promise and today.

5. Bottleneck-removal experiment

Target the queue found in §4: the review-approval wait.

- **Hypothesis:** Because most strings carry no legal/compliance risk, defaulting reviews to **optional** (reviews optional-by-default, with an override to *require* review on flagged products/keys) removes the review queue as the dominant wait for the majority of changes — without letting a required review be skipped.
- **Test:** For 2 weeks on 3 pilot products, ship reviews optional-by-default + override. Measure author→consumed **lead time** (ADO lead-time widget) and the O4 guardrail query (`required review bypassed = 0`). A/B against 3 control products still on always-review.
- **Success criterion:**
  - **Baseline:** p50 lead time ≈ **1.5 days** (review-required flow today).
  - **Target:** p50 lead time ≤ **15 min** for non-review strings; ≥ 80% of changes take the no-review path.

- **Guardrail (must hold):** strings on override-required keys that publish without review = **0**; reject/hot-fix rate no worse than control.
- **Window:** 2 weeks; decision to roll out to all products at the end.

Model answer (facilitator notes — ~1 page)

What "good" shows here. A strong quad's DP-light makes the outcome/output distinction land: O1 is *user-visible* (text change goes live in minutes), not "publish endpoint shipped." The leading indicator (p50 author→consumed lead time) is measurable inside a week and ties straight to the SLOs (≤60s propagation, ≤200ms delivery), so §1 and the TCD/TMD SLOs are the same numbers, not two disconnected lists. Weak versions list outputs ("built the delivery API") as outcomes, or pick a lagging indicator (quarterly adoption) that can't inform a weekly review.

Guardrails are the tell. Each outcome carries a don't-game-it line: O1 can't be won by publishing fast into a stale cache (propagation must hold); O2 can't be won by skipping review (bypass count must stay 0). O4 exists purely as the counter-outcome — speed must not cost governance. Coach quads who wrote three outcomes but no guardrails: ask "how would a cynical team hit that number without delivering the value?"

Backlog traces to the slice. The Epic→Feature→Story tree is the same release-1 slice as every other artifact in this folder, and every story tags its Capability Story (CS3/4 authoring, CS7 delivery, CS11/12 review, CS15 translation) so the chain back to the PRD is checkable. Immutable keys and the Redis cache are correctly modeled as cross-cutting constraints realized *inside* stories, not padded out as fake stories. In ADO terms they'd expect States, Story Points, Area Path, and Tags — a plausible board, not just a bullet list.

Tracking closes the loop. The output→outcome→indicator triples each name a concrete ADO instrument (lead-time widget, saved query, CFD). The CFD earns its place: the review column is exactly where WIP will pile up, which is the same queue §4 and §5 attack — the plan is internally consistent across three sections.

The experiment is honest. It has a real baseline (~1.5 days), a real target (≤15 min, ≥80% no-review path), a hard guardrail (0 required-review bypasses), and a bounded window (2 weeks, A/B'd). It removes the actual bottleneck the value stream exposes rather than a convenient one, and it's directly falsifiable. That is the bar: hypothesis, test, and a measurable definition of "worked." A quad that instead "experiments" on cache propagation isn't wrong — but they must show the queue is the propagation step, and §4 shows it isn't.